CentraleSupélec **Computational Fluid Dynamics**

Project 3

Advection-Diffusion of a Scalar Field

– Thermal mixing –

Turbulence plays a central role in transporting heat and mass in many industrial processes. Examples include chemical reactors, heat exchangers and combustion systems. Turbulence improves the mixing of reactants and mitigates the risk of structural damage from high-temperature zones. In this project, we will study a simplified example of turbulent mixing: the transport of temperature in a swirling velocity field.

1 Swirling flow

In real turbulent flows, a wide spectrum of vortices of various sizes coexist and interact. Capturing them all would require solving the three-dimensional Navier-Stokes equations over the range from the dissipative Kolmogorov scale to the integral scale. However, this approach is impractical for this project due to the enormous computational resources it demands. Instead, we focus on a simplified model that retains the essential features of a turbulent flow: advection and diffusion of conservative quantities in a swirling environment.

We consider a fluid in a two-dimensional square box of size $L = 0.3$ m. We model the flow as a single macroscopic vortex. The Taylor-Green vortex provides an analytical solution to the equations of two-dimensional incompressible flow, which we will use as a fixed advection field for energy transport.

In the inviscid limit (Euler equations), the velocity field for a Taylor-Green vortex is:

$$u(x, y) = -U \sin(\pi x/L) \cos(\pi y/L) \tag{1}$$

$$v(x, y) = U \cos(\pi x/L) \sin(\pi y/L) \tag{2}$$

The amplitude U is set to 0.1 m/s. The velocity field $(u(x, y), v(x, y))$ is prescribed and remains fixed in time; only the temperature field $T(x, y, t)$ evolves. The velocity field is shown in Fig. 1.

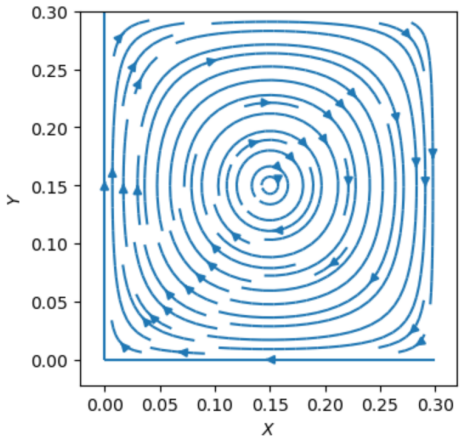


Figure 1: Velocity field (streamlines) in the Taylor-Green vortices

2 Transport of energy

Enthalpy $h = c_p T$ is transported by the fluid motion (advection) and thermal conduction (diffusion). Therefore, the temperature satisfies the advection-diffusion equation:

$$\frac{\partial T}{\partial t} + u\frac{\partial T}{\partial x} + v\frac{\partial T}{\partial y} = D\left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}\right)$$

where u and v are the gas velocities in the x and y directions, respectively. $D = \lambda/(\rho c_p) = 2.5 \times 10^{-5}$ m^2/s is the thermal diffusivity corresponding to the thermal conductivity $\lambda = 0.03$ W/m/K.
The initial temperature field is defined as follows:

$$T(x, y, t = 0) = \begin{cases} T_1 & \text{if } x < L/2 \\ T_2 & \text{otherwise} \end{cases}$$

with $T_1 = 400$ K and $T_2 = 300$ K.

The temperature field is subject to the following boundary conditions: adiabatic on the left ($x = 0$), bottom ($y = 0$), and top ($y = L$) boundaries, and isothermal at temperature T_2 on the right boundary ($x = L$).

3 The problem

We are interested in the evolution of the temperature field from $t = 0$ s to $t_{\max} = 40$ s. In particular, we focus on the following quantities of interest:

- The time evolution of the temperature at the point $x = 3L/4$ and $y = 3L/4$.
- The integrated heat flux Φ_{w} on the right boundary at time $t = t_{\max}$:

$$\Phi_{\mathrm{w}} = \int_{y=0}^{y=L} \left(-\lambda \frac{\partial T}{\partial x}\right)_{x=L} \mathrm{d}y$$

- The maximum integrated heat flux on the right boundary over time $\Phi_{\mathrm{w}}^{\max}$.

4 Mandatory elements of the project

The purpose of this project is to apply the concepts from the course on solving hyperbolic/parabolic systems. You are requested to:

- Implement three convergent numerical schemes of different orders. You must present each scheme and explain its numerical properties.
- Present the results obtained for a 65×65 mesh.
- Assess the mesh convergence of your simulations by measuring the spatial convergence order of each method.
- Determine $\Phi_{\mathrm{w}}^{\max}$ and provide a practical confidence interval based on the mesh sensitivity study.
- Comment on the performance of these methods in terms of computation time for equivalent solution precision.

5 Project track assisted with AI

If you wish to use an AI assistant, here are some guidelines to help you get started.

5.1 Preliminary work without AI

The first step does not require a laptop or AI. We recommend that members of the group discuss the problem together and write their solution strategy down on paper. You should have a clear understanding of this strategy before you start coding. Consult the textbook for the three numerical methods you will implement. Then write down the pseudocode for each method, along with the constraints on the time and space steps. Draft your code architecture on paper. For example:

```
The code is structured in a single file, `advection_diffusion.py`.
It contains the following functions:
i.  main(): The main function initializes the simulation's global parameters
    (mesh, time steps, and array allocation) and runs a time loop which
    contains:
    * calls to functions that update the temperature field
      (update_phi_method_1, update_phi_method_2, or
      update_phi_method_3). Repeat this for each iteration of the time loop.
    * calls to functions that perform post-processing (probe_phi,
      plot_results). Repeat this every 100 iterations of the time loop (or
      as needed).
ii. update_phi_method_1(phi_n): Updates the temperature field using numerical
    method 1. It takes the temperature field at time n as input and returns
    the field at time n+1.
iii. update_phi_method_2(phi_n): Same as (ii), but for numerical method 2.
iv. update_phi_method_3(phi_n): Same as (ii), but for numerical method 3.
v.  probe_phi(phi): A post-processing function that extracts or computes
    quantities of interest from the current temperature field.
vi. plot_results(phi): Displays quantities of interest on the current
    temperature field.
```

5.2 Requirements

Developing code requires a suitable working environment. The reference environment is Unix/Linux.

5.2.1 MacOS and Linux users

If you use macOS or Linux, you already have a suitable Unix-like environment for this project.
Install the `mamba` package manager from Miniforge (`https://conda-forge.org/download/`). Right-click on the link corresponding to your operating system and select `Copy Link Address`. Then, open a terminal and run the following command:

```
# The following command downloads and installs Miniforge, which includes
    Mamba.
# Make sure to update the link and filename if necessary.
curl -L -O [LINK].sh # update with the link you copied
bash [FILENAME].sh # update with the filename you downloaded
```

We recommend using a code editor such as Visual Studio Code (VS Code), which includes an integrated terminal.

5.2.2 Windows users

If you use Windows, we recommend creating a Linux environment on your computer. Here are the reasons why:

1. Windows is not optimized for code development. Managing dependencies and compiling code can be very challenging and time-consuming. This can result in a significant loss of time and effort. Unix-like environments provide a more stable and predictable development experience. They are also the standard environment for scientific computing and software development, so it is best to get used to them.

2. It allows you to use the same tools and commands as in Unix-like systems.

3. Installing a Linux environment on Windows is easy thanks to the Windows Subsystem for Linux (WSL) (`https://learn.microsoft.com/en-us/windows/wsl`). You can install a Linux distribution, such as Debian, Ubuntu, or Fedora, in just a few minutes. Once installed, you can use the Linux terminal to run commands and install packages. The environment can be integrated with Visual Studio Code (VS Code) for a seamless development experience.

Below are the steps to install a Linux environment on Windows using WSL.

Install Debian with WSL

1. Open **Powershell** as **Administrator** (right-click on the Start button > *Windows Terminal (Admin)* or *Powershell*).

2. Run the installation command:

```
wsl --install -d Debian
```

3. Restart your computer if prompted to do so.

4. Upon rebooting, a window will automatically open asking you to create a **username** and **password** for your Debian system.

Configure your Debian system By default, a minimal Debian installation comes with only essential components. You need to install additional packages as needed. Open a terminal in Debian and run the following commands:

```
sudo apt update
sudo apt install -y curl
# install curl to download files, this is necessary for installing
# Mamba/Conda with Miniforge
```

Now, you need to install the `mamba` package manager from Miniforge (`https://conda-forge.org/download/`). Visit the website and identify the correct download link for your Debian system. Right-click on the link and select `Copy Link Address`. Then, open a Debian terminal and run the following command:

```
curl -L -O [LINK] # update with the link you copied
bash [FILENAME].sh # update with the filename you downloaded
```

Launch Debian in VS Code We recommend using Visual Studio Code (VS Code) to develop your code. VS Code has a built-in terminal that allows you to run commands in your Debian system.

1. Open **Visual Studio Code**.
2. Install the official **WSL** extension (search for "WSL" in the Extensions tab or press `Ctrl+Shift+X`).
3. Open the Command Palette using `Ctrl + Shift + P` (or `F1`).
4. Type and select: `WSL: Connect to WSL` (then choose **Debian** if a list appears). VS Code will configure the remote environment (this may take a few seconds the first time).
5. Once connected (you will see the green `WSL: Debian` indicator at the bottom-left corner), open an integrated terminal via `Ctrl + ` ` (or through the menu *Terminal* > *New Terminal*).

You are now in a full Debian terminal, ready to code! From now on, all commands must be run in the Debian terminal.

5.3 Project structure and environment management

Having a clear project structure is essential for maintaining the codebase and collaborating effectively. Create a folder named `cfd_advection_diffusion` for your project. A typical Python project can be organized as follows:

```
cfd_advection_diffusion/
├── README.md # project description
├── environment.yml # Conda environment definition
├── pyproject.toml # project metadata
├── advection_diffusion/
│   └── advection_diffusion.py
├── tests/
├── results/
├── AGENTS.md # shared by group members
└── skills/
    └── skills.md
```

`README.md` describes the project. You can use it to document the purpose, usage, and other relevant information about the project. `environment.yml` defines the Mamba/Conda environment and its dependencies, allowing everyone in the group to recreate the same setup. `pyproject.toml` contains project metadata and configuration of tools. Folder `advection_diffusion/` contains the source code. `AGENTS.md` contains instructions for the AI agent. We will create those files as you progress through the project.
Ask an AI to generate the `environment.yml` and `pyproject.toml` files. Below is an example of a prompt:

```
Write an 'environment.yml' file to create a Mamba/Conda environment named
   'cfd-env' with the following dependencies: Python, NumPy, Matplotlib,
   SciPy, and Numba. Remind me how to create the 'cfd-env' environment on my
   machine, then how to run a script in that environment.
```

Copy the result of the AI's response and paste it into the `environment.yml` file. A typical example looks like this:

```
name: cfd-env
channels:
```

```
  - conda-forge
dependencies:
  - python
  - numpy
  - scipy
  - spyder
  - matplotlib
  - numba
```

Then, create the `cfd-env` environment on your machine by opening a Linux/Unix terminal and running the command suggested by the AI. The command is usually:

```
mamba env create --file environment.yml # if you have Mamba installed
conda env create --file environment.yml # if you have Conda installed
```

You just created a Mamba/Conda environment named `cfd-env` with the required dependencies. To run a script with the terminal in this environment, activate it first:

```
mamba activate cfd-env
```

Now you can run your Python script in the `cfd-env` environment. For example:

```
python advection_diffusion.py
```

The Python used will be the one installed in the `cfd-env` environment, not the one installed on your system. The script will have access to the dependencies installed in the `cfd-env` environment: `numpy`, `scipy`, `matplotlib`, and `numba`.
To deactivate the environment, run:

```
mamba deactivate # or conda deactivate
```

To run the script in the `cfd-env` environment with VS Code, follow the instructions here: `https://code.visualstudio.com/docs/python/environments#_select-an-environment`.

5.4 Guiding the AI

You will now give the AI instructions to help you implement the numerical schemes based on the strategy you developed in §5.1. There are two cases to consider: either you have an AI agent installed on your computer or you only have access to an AI chat interface. The following two subsections detail each case. Read the first subsection even if you don't have an AI agent because it contains information necessary for understanding the second subsection.

5.4.1 You have an AI agent (Claude Code, OpenCode) installed on your computer

Provide the AI with your strategy. The usual way to do this is to edit the `AGENTS.md` file. The AI agent will read this file and follow its instructions when assisting you.
Note: `AGENTS.md` is a cross-platform format compatible with all AI agents (`https://agents.md/`).
Open `AGENTS.md` and add:

- All your handwritten notes from §5.1, including your strategy, pseudocode, and architecture.
- Specific instructions such as:

```
Implement the numerical method [method name] in a highly readable style that
follows the pseudocode as closely as possible:
```

```
- Use loops over indices i and j to traverse the mesh.
- Avoid slicing (e.g., phi[1:-1, 1:-1]) to make the code easier to read.
- Add the @numba.njit decorator to bottleneck functions to speed them up.
- Add a NumPy-style docstring to every function.
- If you need to run the code for testing, activate the cfd-env environment
  first: conda activate cfd-env.
  # optional, for those using an AI agent on their computer
- [...] # Add your own preferences here!
```

You now have an `AGENTS.md` file. Unfortunately, it may not follow the standard format, which could make it more difficult for the AI to understand. It needs to be cleaned up and formatted correctly. Copy its contents, then use the following prompt:

```
Here is the content of my AGENTS.md file. Clean it up so that it
follows the standard format for AGENTS.md files?
[PASTE THE CONTENT OF YOUR AGENTS.md FILE HERE]
```

Review the cleaned-up content and make sure it follows your strategy. If it does not, revise it. If it does, update the `AGENTS.md` file with the cleaned-up content.
Share the `AGENTS.md` file with all group members so that their AI agents' work remains consistent (as much as possible) and merging their contributions is easier.

5.4.2 You do not have an AI agent installed on your computer

If you only have access to an AI chat interface, find the "System Prompt" tab in the preference settings and add the same instructions you would have put in the `AGENTS.md` file. Whenever you ask the AI a question, these instructions will be added to the beginning of your prompt and guide its response. All group members should use the same system prompt so that their work stays consistent and their code can be merged more easily.

5.5 Work distribution

Divide tasks (i), (ii), (iii), (iv), (v), and (vi) described in §5.1 among the group members. Each member should have the same `AGENTS.md` file and/or system prompt so that everyone's work remains consistent and their contributions are easier to merge.
For example, here is a possible division of work among three students:

- **Student 1**: (i), (ii)
- **Student 2**: (iii), (v)
- **Student 3**: (iv), (vi)

Despite this division of responsibilities, every member should understand all the functions.